

Highlights

skycell: a cost-based HEALPix index for sky positions in PostgreSQL

Jesús Salgado, Michele Delli Veneri

- A HEALPix B-tree index for PostgreSQL answering cones, polygons and cross-matches
- A cost model sets each covering's depth from local density and index range price
- Faster than pgSphere at every cone radius from 1 arcsec to 3 deg, real Gaia DR3 too
- Cross-matches 20–33% faster than q3c_join, from an index a third of pgSphere's size
- Speaks ADQL; one column holds circles and polygons, read and written as IVOA STC-S

skycell: a cost-based HEALPix index for sky positions in PostgreSQL

Jesús Salgado^{a,*}, Michele Delli Veneri^a

^a*SKA Observatory, Jodrell Bank, Lower Withington, Macclesfield, SK11 9FT, United Kingdom*

Abstract

Astronomical archives answer positional queries from relational databases, and in PostgreSQL two extensions dominate: Q3C, which keys sources by a cube quad-tree index in a B-tree, and pgSphere, which stores spherical geometry in a GiST tree and also indexes regions. We present **skycell**, a PostgreSQL extension that keeps the integer key and the B-tree yet answers cones, polygons, cross-matches and stored footprints in the archives' own ADQL vocabulary. Positions are keyed by their order-29 HEALPix cell, and a query region is covered by cells whose depth is chosen by a cost model: the price of an extra index range against the false-positive rows it removes, with local source density read from the histogram PostgreSQL already keeps. Cones and cross-matches are answered by a planner custom scan that walks the covering's ranges, polygons by a rewrite into range conditions, and stored regions by a GiST operator class. Coverings are validated by brute force over millions of points. In randomized paired trials on PostgreSQL 18.6 with 10- and 50-million-row catalogues, including real Gaia DR3 positions, **skycell** is faster than pgSphere at every cone radius from 1" to 3°, by 4–33% with a warm cache and up to 2.7 times cold. It cross-matches 20–33% faster than Q3C's `q3c_join` and three times faster than pgSphere, and answers convex polygons 2.7–4.4 times faster than pgSphere, from an index a third the size. One index serving points, regions and cross-matches through ADQL makes **skycell** a strong default for an archive's positional layer.

*Corresponding author

Email addresses: `jesus.salgado@skao.int` (Jesús Salgado),
`michele.delliveneri@skao.int` (Michele Delli Veneri)

Keywords: Astronomical databases, Spatial indexing, HEALPix, PostgreSQL, Cross-matching, Virtual Observatory, ADQL

1. Introduction

Almost every question asked of an astronomical archive is positional: what is within this cone, which observations cover this field, which of my sources match that catalogue. In the Virtual Observatory these arrive as ADQL [13] through the Table Access Protocol [4], and the service must answer them with an index. The scale is set by archives already in operation: the ESA Gaia Archive [15, 16] serves order 10^9 sources and lists cross-match among its server-side capabilities.

Two PostgreSQL extensions carry most of this traffic. Q3C [10] projects the sphere onto a cube, subdivides each face with a quad-tree, and numbers the cells along a space-filling curve, so each source is one 64-bit integer in a B-tree. pgSphere [1] adds spherical types indexed with GiST over bounding volumes, so regions can be stored and queried as data. Neither idea is new – the Hierarchical Triangular Mesh [11] keyed positions by a subdivided polyhedron, and the zones algorithm [9] arranged a catalogue for cross-matching – but catalogues have grown, and most queries now arrive through a generated layer [e.g. 3] rather than being hand-written.

The two designs trade off: a linear key in a B-tree is compact and cheap to probe, but a region must first become a set of key ranges, whose quality depends on how finely the region is cut; a tree of bounding volumes needs no such step but has larger, overlapping keys. This paper presents *skycell*, which keeps the integer key and treats the cut as a cost question answered from two quantities the database already knows: local source density and the price of an index range. Section 2 describes the index, Section 3 the region index, Section 4 its correctness, Section 5 the ADQL surface, Section 6 the performance comparison, and Section 7 concludes.

2. The index

2.1. An equal-area integer key

A position is keyed by its order-29 HEALPix cell in the nested scheme [8] ($12 \cdot 4^{29} \approx 3.5 \times 10^{18}$ cells of about 0.4 mas), stored as a 64-bit integer, so a B-tree orders keys along the HEALPix curve. Three properties follow.

Cells are *nested*: the order-29 descendants of a cell p at order k are exactly the contiguous interval

$$[p \cdot 4^{29-k}, (p+1) \cdot 4^{29-k} - 1], \quad (1)$$

so a region covered by cells of any orders becomes a union of integer intervals a B-tree answers with range scans. Cells are *equal area*, so keys per interval are proportional to source density per solid angle: the **ANALYZE** histogram is a crowding map with no extra machinery (Section 2.2). And cells are *standard* – the numbering underlies Gaia’s `source_id` [7] and IVOA MOC maps [6] – so keys, coverings and footprints are interoperable.

2.2. Coverings decided by cost

A query region is covered by cells whose union contains it; the covering becomes range conditions on the key, and rows inside the covering but outside the region are removed by an exact test. Cutting finer means fewer false positives and more ranges. With cells of size s covering a region of radius r and local source density ρ ,

$$C(s) = n_{\text{ranges}}(s) c_{\text{range}} + \rho (A_{\text{cover}}(s) - A_{\text{region}}), \quad (2)$$

both terms in rows: the second is the expected false positives, and c_{range} converts one additional index range into the false-positive rows it is worth avoiding. A covering follows the region’s boundary, so for $s \ll r$ the ranges grow as r/s while the wasted area grows as the perimeter times s ; writing $A_{\text{cover}} \simeq \pi(r + \beta s)^2$ and $n_{\text{ranges}} \simeq \gamma r/s$ and minimising Eq. (2) gives

$$s_{\star} = \sqrt{\frac{\gamma c_{\text{range}}}{2\pi\beta\rho}}, \quad (3)$$

independent of region size: sky density and range price alone decide how finely to cut. For $s \gtrsim r$ the balance differs, so the implementation evaluates Eq. (2) at all 30 orders and takes the cheapest. The same 0.25° cone illustrates both regimes: at high Galactic latitude, 11 sources leave almost nothing to exclude, so the model buys one range of large cells (5.6 times the cone’s area); in the bulge, 1183 sources make false positives cost more than extra ranges, so it cuts two orders deeper (8 ranges, 1.3 times the area). Two guards keep the model safe against its own inputs: a partly covered cell may not exceed 64 times the region’s area, bounding the damage from a cluster the histogram underestimates, and gaps between ranges are merged when the rows they hold cost less than the range they save.

The range price.. c_{range} is not a free parameter: it is the ratio of two costs PostgreSQL already models, one more index range against one false-positive row. A range costs a B-tree descent plus two pages – the leaf it lands on and the heap page where the scan stops running sequentially – and a false row its index entry, heap tuple, exact test and its share of a heap page, each page priced at $p = f 50 c_{\text{op}} + (1 - f) c_{\text{page}}$ by where it is expected to come from:

$$c_{\text{range}} = \frac{\lceil \log_2 N \rceil c_{\text{op}} + 50(H + 1) c_{\text{op}} + 2p}{c_{\text{tuple}} + c_{\text{itup}} + k c_{\text{op}} + p/n_{\text{pp}}}, \quad (4)$$

with N rows, H the B-tree height, n_{pp} rows per heap page, k the operators of the exact test, the c the planner’s own cost factors, and $f = \min(1, \text{shared_buffers}/\text{relpages})$ the fraction of the relation the buffer pool can hold. For a 10-million-row catalogue this is 34 rows with the relation in the pool and 87 with none of it there; a wide observation row, with fewer rows per page, is cut finer. The cost curve is flat near its optimum – across four decades of c_{range} the median query time moves by under 25% between 3 and 300, and the order chosen is within about 3% of the best – so deriving the price per relation costs nothing in speed and leaves nothing to tune.

The density estimate.. **ANALYZE** stores, for the indexed expression, an equi-depth histogram of $B + 1$ bounds $h_0 < \dots < h_B$ such that each bucket holds about N/B rows. Because the key is an equal-area cell index, the rows in a key interval $[a, b]$ are estimated by linear interpolation within the buckets it spans,

$$\hat{n}(a, b) = \frac{N}{B} \sum_i \frac{|[a, b] \cap [h_i, h_{i+1})|}{h_{i+1} - h_i}, \quad (5)$$

and the density in rows per steradian as

$$\hat{\rho}(a, b) = \frac{\hat{n}(a, b)}{(b - a + 1) A_{29}}, \quad (6)$$

with A_{29} the area of a leaf cell, evaluated over the cell containing the region’s centre at the order whose cells are about four times the region’s size: large enough to hold a density, small enough to be local. A cluster smaller than one bucket is averaged away, which the area guard bounds; Section 6.6 measures the estimate on real positions.

Classifying cells.. A cell is classified against the region as inside, outside or straddling its boundary. The outcomes are not symmetric: a cell wrongly called *outside* silently loses rows, while *inside* and *straddling* cells only produce a superset the exact test trims. Only the outside decision therefore needs a bound. The one used is

$$R_{\text{cell}}(k, p) = \min(\text{maxpixrad}(k), \max_i \angle(\mathbf{c}_{k,p}, \mathbf{v}_i)), \quad (7)$$

with $\mathbf{c}_{k,p}$ the cell’s centre, $\mathbf{v}_i$ its corners, and the first term the analytic worst case of the reference HEALPix implementation. A cell is outside a cone of centre $\mathbf{q}$ and radius r when $\angle(\mathbf{q}, \mathbf{c}_{k,p}) - R_{\text{cell}} > r + \epsilon$ (per bounding half-space for a polygon), with $\epsilon = 2 \times 10^{-13}$ rad covering accumulated floating-point error, five orders of magnitude below the leaf cell. The inside test uses the cell’s own corners and edge midpoints, since optimism there costs false positives, never rows.

2.3. Answering a query

The extension is written in C against the PostgreSQL server API, with a planner support function on each spatial predicate. A cone – `skycell_cone`, the Q3C-shaped `skycell_join` and `skycell_radial_query`, or ADQL’s own `point <@ circle` – is left as one clause whose selectivity the support function supplies, and answered by a planner *custom scan*: a path hook offers it wherever a B-tree on the cell expression can serve the clause, costed as PostgreSQL costs the same work. The node computes the covering and walks its ranges in one of two modes, chosen from the column’s correlation as PostgreSQL chooses between an index and a bitmap scan: one index scan per range in cell order when the heap follows the cell order, or every range’s TIDs into one bitmap, each heap page read once, when it does not. Rows are rechecked with the exact test. A cone whose centre comes from another relation’s row – a cross-match – gets the same node parameterized by that row on the inner side of a nested loop, each probe covered with the ranges its own cone needs; its row estimate is measured at plan time on 32 sampled probes, since a density model cannot know that cross-match targets sit on catalogue sources. Polygons and boxes are instead rewritten before path generation into a disjunction of range conditions on the key plus the exact test, whose selectivity the planner estimates from the same histogram. The density model, index identity and coverings are cached per backend.

Ingestion and maintenance. The index is an ordinary B-tree on an expression: one 8-byte key per insert, no method-specific rebalancing. Because the covering reads the histogram, statistics matter more than for a GiST index; the area guard bounds the damage from a stale one, and **ANALYZE** on a 10-million-row catalogue takes about a second.

3. The region index

Regions as stored objects. **skyregion** is one type tagged by kind rather than a family of per-shape types, so a single footprint column holds circles and polygons together – as an ObsCore **s_region** column does, mixing single-pointing circles with polygons. pgSphere’s **scircle** and **spoly** are separate types, so a mixed-shape column needs two columns in its place. A footprint can be covered by a few cells written in the MOC standard’s **NUNIQ** encoding and stored one per row in an ordinary B-tree; the footprints containing a position are those whose cell is one of its ancestors, a short list of equality lookups.

A GiST opclass family. A **skyregion** column can instead carry its own GiST index, serving **&&** and both containment operators directly. Its default key is an axis-aligned 3D box rounded outward to **float4** (24 bytes, pgSphere’s own GiST representation), sound because the recheck is exact whatever the key’s precision: a looser box costs false positives, never rows. A key of a few spherical caps, exact for a circle at any radius, remains available for columns mixing very different region scales.

4. Correctness

Proposition. Let R be a cone or convex polygon and let $\mathcal{C}$ be the covering computed for it. If (i) every point of a HEALPix cell at order k lies within $\text{maxpixrad}(k)$ of that cell’s centre, and (ii) every angle computed by the classification carries an absolute error below ϵ , then every order-29 cell that intersects R lies in one of the ranges of $\mathcal{C}$.

Proof sketch. The descent starts from cells whose union is the whole sphere and, at each step, keeps a cell, drops it, or replaces it by its four children, so the kept union shrinks only where a cell is dropped, which happens only when it is classified outside. By (i), every point of cell (k, p) lies within

R_{cell} of its centre, so (ii) and the triangle inequality make $\angle(\mathbf{q}, \mathbf{c}_{k,p}) - R_{\text{cell}} > r + \epsilon$ sufficient for the whole cell to miss the cone; the polygon case applies the same argument per bounding half-space. Truncation (range cap, area guard, step budget) only stops refinement, keeping a cell whole, so it cannot remove one either. $\square$

Condition (i) is a property of HEALPix (the reference maxpixrad is analytic; verified over 1.6×10^6 sampled boundary points, largest observed ratio 0.9999); (ii) is an assumption about floating-point arithmetic, with ϵ five orders of magnitude above the plausible accumulated error. Because a missed cell silently loses rows, the implementation is also checked by exhaustion: the HEALPix layer (known centres, round trips at all 30 orders, nesting, equal area), and the covering against a 2-million-point clustered catalogue for cones to 120° and convex polygons, 30 000 adversarial cones (poles, zone boundaries, face edges, radii $0.02 \text{ mas} - 10^\circ$) and 7266 adversarial polygons with 869 117 points – all asserting zero false negatives. SQL regression suites, run on PostgreSQL 16 and 18, require every indexed answer to equal a sequential scan with the exact predicate, and the custom scan to return the same rows as the range rewrite in both of its modes, for constant cones, cross-matches and every spelling of the predicate.

5. The ADQL surface

The extension exposes ADQL’s own vocabulary: types `skypos` and `skyregion` and functions `point`, `circle`, `box`, `polygon`, `contains`, `intersects`, `distance`, `area`, `coord1`, `coord2` and `centroid`, so a translator emits the standard’s spelling rather than a dialect (Table 1). The operator `point <@ circle` is the indexable form of `CONTAINS`, answered by the custom scan whether the circle is a constant or comes from another table’s row, which makes an ADQL cross-match as fast as the native one. `skyregion`’s text representation is IVOA STC-S [14], the syntax an ObsCore `s_region` column already carries, so such a column casts from and prints back as the archive’s own text. For SQL migrating from Q3C, `skycell_radial_query` and `skycell_join` take Q3C’s argument lists unchanged. The IVOA UDF registry’s epoch-propagation and HEALPix functions are implemented with the rigorous propagation of the Hipparcos catalogue [5], so a proper-motion-aware cone search is an indexable cone widened by the largest motion of interest plus an exact test on the propagated position.

Table 1: What a translator emits for the ADQL constructs where the pgSphere-plus-Q3C pair and `skycell` differ most.

ADQL	pgSphere + Q3C	skycell
POINT/CIRCLE/ POLYGON	spoint/scircle/spoly	point/circle/ polygon
CONTAINS(POINT, CIRCLE)	q3c_radial_query <i>or</i> pos <@ scircle	point <@ circle
INTERSECTS(r_1 , r_2)	$r_1 \ \&\& \ r_2$ (no Q3C region type)	$r_1 \ \&\& \ r_2$
cross-match	q3c_join, a separate function	point <@ circle in the join

6. Performance

6.1. Corpora, protocol and hosts

Three 10-million-row catalogues answer the objections each previous one invites. The *designed* corpus is synthetic, with a density contrast resembling an optical survey (35% uniform, 45% Galactic disk, 12% bulge, 8% in 200 clusters). The *Gaia* corpus places positions in proportion to the Gaia DR3 source counts of every order-9 HEALPix cell (3 145 728 cells of 0.11°), the real sky’s crowding down to that scale; a 50-million-row version tests scale. The *real* corpus is ten million real Gaia DR3 positions, with every scale of clustering. An ObsCore-shaped relation of observations grouped by field, at three sizes, stands for an archive’s observation table.

Q3C, pgSphere and `skycell` each hold the same rows in their own index, heap ordered by their own key. Every query is a trial: the methods answer it back to back in random order, five times, and the analysis is paired – the ratio between methods is formed per query, so its difficulty cancels – with a percentile bootstrap interval (4000 resamples); a ratio is called a difference only when its interval excludes 1. *Warm* follows an untimed pass; *cold* is the first touch after a server restart and a page-cache drop. Cone radii span $1''$ – 3° with 16–400 queries each, half centred on catalogue positions and half uniform on the sky.

Measurements use PostgreSQL 18.6, Q3C 2.0.5 and pgSphere 1.5.2 with one configuration (2 GB `shared_buffers`, `random_page_cost` 1.1, JIT and parallelism off). Cones, scale, cross-matches, the ObsCore relation and the density estimate were measured on a 4-vCPU x86-64 cloud virtual machine with local flash storage (a cold page read costs 0.06–0.1 ms); polygons, stored

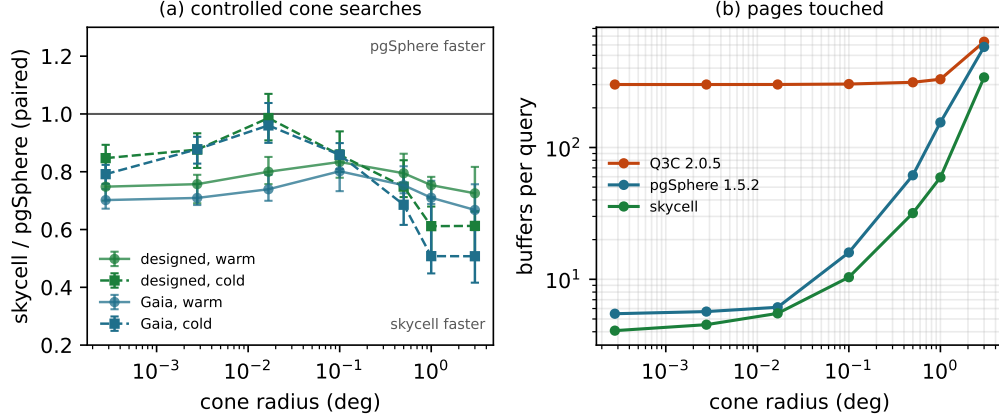


Figure 1: (a) Paired ratio of `skycell` to `pgSphere` against cone radius, 95% bootstrap intervals, synthetic corpora, warm and cold; below the line `skycell` is faster. (b) Buffer pages touched per query (mean, designed corpus).

regions, the memory test and the TAP service on an Apple M3 Pro laptop, under Docker.

6.2. Cone searches

Table 2 and Fig. 1 give the primary result. Warm, `skycell` is faster than `pgSphere` at every radius on all three corpora, every interval excluding 1: by 17–33% on the two synthetic ones and by 4–31% on real positions. At $1''$ it plans in 0.036 ms against `pgSphere`’s 0.029 and wins in execution, 0.022 ms against 0.040, reading 4.1 pages to `pgSphere`’s 5.5. Cold it is faster or level everywhere: the margin grows with radius to a factor 1.6–2.1 at 1° and 3° , because it touches a third of `pgSphere`’s pages there (59 against 155 on the designed corpus, 85 against 282 on the Gaia one). On real positions it plans in at most 0.004 ms more than `pgSphere` and executes in 0.52–0.82 of its time, reading fewer pages at every radius; cold, it is faster by a factor 1.5–2.1 from $6'$ up and level or slightly faster below, where a first-touch query reads only two or three pages. On flash storage a saved page is worth about 0.1 ms; on a slower disk each would count for more.

Against Q3C, `skycell` takes 0.02–0.58 of the time, a comparison of *implementations*: `q3c_radial_query` 2.0.5 expands every call into about 100 bitmap index scans whatever the radius, spending some 2.7 ms planning them. A covering-aware radial query in Q3C would close most of that gap;

Section 6.4 compares the approaches where Q3C carries no such expansion.

6.3. Scale and index size

Ten million rows is 0.55% of Gaia DR3, so the protocol was repeated at 50 million rows, where pgSphere’s 3439 MB index exceeds the 2 GB buffer pool while **skycell**’s 1071 MB B-tree still fits (Table 3). Warm, **skycell** stays 14–31% faster at every radius. Cold, its lead grows with size from 30’ up, to a factor 1.9 at 30’, 2.7 at 1° and 2.4 at 3°, as the page gap widens (188 pages against pgSphere’s 642 at 1°). Its row estimates stay within a factor 1.3 of the truth at every radius, where pgSphere’s are off by up to 8.7 and Q3C’s by up to 2.4.

The per-row index cost is flat between 10^7 and 5×10^7 rows, so it extrapolates to Gaia DR3’s 1.81×10^9 sources (Table 4). The comparison that matters is between deployments: an archive answering both cone searches and cross-matches today carries pgSphere plus Q3C, whose `q3c_join` is 2.4–2.6 times faster than pgSphere at cross-matching (Section 6.4); **skycell** answers both, faster, from one index. At DR3 scale that is 38 GiB against 160 GiB, the difference between a 64 GiB and a 256 GiB server. Where the buffer pool cannot hold every index, the smaller one wins: in a 3 GiB container (768 MB of shared buffers) holding a 22 GB, 20.5-million-row ObsCore-shaped relation, whose GiST index is 1.5 times the pool and **skycell**’s 0.57 times, **skycell** answered a stream of 2° cones sweeping the sky 4.2–4.8 times faster.

6.4. Cross-matching and polygons

Cross-matching a user’s target list is the operation the Gaia archive lists among its server-side capabilities and Q3C’s own speciality, so it gets the cone searches’ protocol, each method warmed on its own query before timing. On real Gaia DR3 positions, at the radii cross-matching uses – optical 1–1.5”, radio up to 5” – with uploads of 10^3 – 10^5 targets, clustered or uniform (Table 5), **skycell**’s join is 0.79 of `q3c_join`’s time and 0.32 of pgSphere’s over all 24 cells, and faster than both in every one. On the resampled corpus, 200 000 probes at 1” and 10”, it is 0.67–0.75 of `q3c_join` and 0.26–0.31 of pgSphere, returning the same rows in every trial. The custom scan covers each probe’s cone with the ranges it needs, 1.2 on average at 1”, reading 4.3 pages per probe. Written with `skycell_cone`, `skycell_join` or ADQL’s `point <@ circle`, the join is the same plan.

Table 2: Cone searches: paired ratio of `skycell` to `pgSphere` with its 95% bootstrap interval. Below 1, `skycell` is faster. **Bold:** interval excludes 1. n : distinct queries at that radius. Real corpus: its own centre draw at the same radii and counts; cold there uses one table per method and fresh centres, so neither method warms the other’s pages.

radius	n	designed corpus		Gaia corpus (resampled)		real Gaia DR3 positions	
		warm	cold	warm	cold	warm	cold
1''	400	0.75 [0.70, 0.79]	0.85 [0.80, 0.89]	0.70 [0.67, 0.74]	0.79 [0.76, 0.83]	0.83 [0.81, 0.84]	0.87 [0.82, 0.94]
10''	400	0.76 [0.69, 0.79]	0.88 [0.81, 0.94]	0.71 [0.68, 0.76]	0.88 [0.83, 0.92]	0.85 [0.82, 0.86]	1.01 [0.94, 1.09]
1'	300	0.80 [0.76, 0.85]	0.99 [0.91, 1.07]	0.74 [0.70, 0.79]	0.96 [0.90, 1.04]	0.90 [0.87, 0.92]	0.98 [0.90, 1.04]
6'	200	0.83 [0.78, 0.88]	0.86 [0.80, 0.94]	0.80 [0.74, 0.86]	0.86 [0.81, 0.90]	0.96 [0.93, 0.99]	0.61 [0.55, 0.65]
30'	100	0.79 [0.75, 0.86]	0.75 [0.71, 0.84]	0.75 [0.73, 0.82]	0.69 [0.62, 0.76]	0.81 [0.78, 0.86]	0.67 [0.60, 0.72]
1°	40	0.75 [0.71, 0.78]	0.61 [0.52, 0.68]	0.71 [0.69, 0.77]	0.51 [0.45, 0.72]	0.80 [0.75, 0.88]	0.53 [0.44, 0.66]
3°	16	0.73 [0.66, 0.82]	0.61 [0.50, 0.66]	0.67 [0.63, 0.76]	0.51 [0.41, 0.60]	0.69 [0.64, 0.75]	0.48 [0.43, 0.54]

Table 3: Paired ratio of `skycell` to `pgSphere` on the Gaia density field at two catalogue sizes. **Bold:** interval excludes 1.

radius	warm		cold	
	10^7	5×10^7	10^7	5×10^7
1''	0.70	0.78	0.79	0.82
10''	0.71	0.78	0.88	0.86
1'	0.74	0.86	0.96	0.98
6'	0.80	0.86	0.86	0.81
30'	0.75	0.75	0.69	0.53
1°	0.71	0.69	0.51	0.37
3°	0.67	0.73	0.51	0.42

Table 4: Index size and build time at 10^7 rows, and extrapolated linearly from the 5×10^7 measurement to Gaia DR3 (1.81×10^9 sources). Last column: the smallest conventional server memory holding the index with room for the heap.

index	10^7 rows		at Gaia DR3 scale		
	MB	build (s)	GiB	build	mem GiB
Q3C	214	5.6	38	15 min	64
pgSphere	685	68.6	122	3.9 h	192
skycell	214	6.0	38	10 min	64
pgSphere + Q3C	—	—	160	4.1 h	256

Convex polygons of 0.05° – 2° take a median 0.28ms against `pgSphere`’s 0.77 and Q3C’s 1.86, a factor 2.7–4.4 on both synthetic corpora, with `pgSphere`’s tail far above its median where a polygon’s bounding volume fits it poorly. Stored footprints follow the same pattern: the MOC-in-a-B-tree arrangement answers 200 474 point-in-region pairs in 0.76s against `pgSphere`’s 0.97s.

6.5. An archive’s observation table

An `ObsCore`-shaped relation – observations in groups of 128 sharing a field centre, clustered by collection and field – was built at three sizes from a stored set of field and query centres drawn from the DR3 density map, and asked the same 30 queries at each (Table 6). `skycell`’s lead grows with the table: by 10^7 rows it is level on the 0.15° cone and faster on every other class, 25% on the 2° cone, where `pgSphere` reads 939 pages to its 591. The one class `pgSphere` wins is the small cone on the smallest table, where its bitmap scan over field-clustered data already reads exactly the pages it needs. On a table that small, the database is not what limits an archive anyway: inside a TAP 1.1 service [2] on a 500 000-row `ObsCore` corpus, the two indexes tie on

Table 5: Cross-matching on real Gaia DR3 positions (10^7): geometric mean over three upload sizes (10^3 – 10^5) and two target distributions of the per-cell ratio of median times, method order random per cell. *Cone*: the join written with `skycell_cone`; *join*: with `skycell_join`, Q3C’s spelling. Below 1, `skycell` is faster; **bold**: faster in every cell.

radius	vs q3c_join		vs pgSphere
	cone	join	join
0.5''	0.81	0.77	0.30
1''	0.77	0.76	0.30
1.5''	0.77	0.78	0.34
5''	0.84	0.83	0.35
all	0.80	0.79	0.32

Table 6: ObsCore-shaped relation at three sizes: total query time of `skycell` as the ratio to pgSphere (pgS.), 30 stored centres, two complete randomized runs pooled; median buffer pages at 10^7 rows. Q05: 0.15° cone; Q06: 2° ; Q07: 0.5° with calibration and time cuts; Q12: an empty $0.7''$ cone.

rows (10^6)	skycell / pgS.			buffers, 10^7	
	0.5	2	10	pgS.	skycell
Q05	1.14	1.07	0.97	32	32
Q06	0.92	0.93	0.75	939	591
Q07	0.94	0.99	0.80	113	89
Q12	0.71	0.68	0.47	7	3

throughput at 1–32 concurrent clients, because the service’s own code costs about 10 ms a request against a fraction of a millisecond in the database.

6.6. The density estimate on real positions

The cost model reads $\hat{\rho}$ from Eq. (6). Table 7 compares it with ρ counted from the same real-position relation: within 5% at Baade’s Window and a factor 1.3 at the Galactic pole, low by a factor 10–50 in a globular cluster core and high by 3.7 at the Galactic centre, as the histogram resolves a cluster while a query sits inside it and averages it away once the query is larger. The consequence is smaller than the error: since $s_\star \propto \rho^{-1/2}$, an order of magnitude in ρ is a factor three in cell size, and the cost curve is flat across much of that. The covering chosen in a cluster core is within 1.05–1.53 of the best fixed order, and within 0.98–1.31 where the estimate is accurate, against 243–830 for a fixed order chosen badly: adaptive resolution is worth far more than the estimate’s error costs.

Table 7: The density estimate $\hat{\rho}$ against the truth ρ on real Gaia DR3 positions, radii 0.01° – 1° : median ratio over radii, then over ten **ANALYZE** samples; *samples*: spread of that median across them; *range*: over all radii and samples.

field	$\hat{\rho}/\rho$	samples	range
ω Cen	0.08	0.06–0.09	0.05–0.54
47 Tuc	0.02	0.01–0.11	0.00–0.94
LMC	0.80	0.46–0.92	0.22–1.26
Baade’s Window	0.96	0.92–0.98	0.45–1.19
Galactic pole	1.27	1.18–1.49	0.48–1.69
Galactic centre	3.71	3.41–3.80	1.95–5.20

Table 8: Region operators on the 50 000-row **fpr** catalogue, 500 probes per operator: time of the **float4** box key as the ratio to pgSphere’s native GiST. Below 1, **skycell** is faster.

operator	box4 / pgSphere
&& (overlap)	0.12–0.17
@> (region,point)	0.75–0.93
@> (region,region)	0.55–0.70
<@ (region,region)	0.26–0.27

6.7. Stored regions

On the real 50 000-row **fpr** footprint catalogue, the **float4** box key beats pgSphere’s native GiST on all four region operators (Table 8), and on isolated single-scale bands it reads 10–25% fewer buffers in 10–20% less time; on a mixed circle and polygon corpus the margin widens, since pgSphere needs a separate index descent per region type.

7. Conclusions

skycell keeps Q3C’s compact integer key and B-tree while answering the region queries pgSphere was built for, and treats a covering’s depth as a cost question – local source density against the price of an index range – rather than a constant. Measured paired against both, it beats each on its own ground. At the cone search, pgSphere’s speciality, it is faster at every radius from $1''$ to 3° warm, by 17–33% on synthetic catalogues and 4–31% on real Gaia DR3 positions, faster or level cold, and further ahead as the catalogue grows; it answers convex polygons 2.7 – $4.4\times$ faster. At the cross-match, Q3C’s speciality, it is 20–33% faster than **q3c_join** on resampled and real positions alike, and about three times faster than pgSphere. It does so from an index a third the size of pgSphere’s that builds 11–23 times faster,

with one cost parameter derived from the relation’s own statistics, and its region GiST beats pgSphere’s on every operator.

The interface matters as much as the speed. `skycell` speaks ADQL’s own vocabulary, with the indexable `point <@ circle` answered by the same scan as its native functions; its stored regions read and write as IVOA STC-S; and one column holds circles and polygons together, which pgSphere cannot. One compact index serving points, regions and cross-matches through the language archives already use makes `skycell` worth considering as the default positional index of a PostgreSQL archive, and in particular as a TAP service’s geometry layer.

Two limits remain. Planning still costs: a covering is computed per query, so on a small table a small cone can favour pgSphere (1.14 at 5×10^5 rows), and at 1" `skycell`’s planning, 0.036 ms, is above pgSphere’s 0.029 even as its execution wins. And the density synopsis is coarse: an equi-depth histogram cannot see a cluster smaller than a bucket, which the area guard bounds rather than fixes; a stored multi-order count map is the natural next step.

CRedit authorship contribution statement

Jesús Salgado: [roles]. **Michele Delli Veneri:** [roles].

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used Claude Code (Anthropic) in order to develop the software, the benchmark harness and the analysis, and to draft and edit the manuscript. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article. The geometric predicates and coverings are checked by the brute-force suites of Section 4 against independent computations over millions of adversarially placed points rather than expected outputs, and every indexed answer in the regression suites must equal a sequential scan with the exact predicate.

Data availability

The extension, its test suites, the benchmark harness and the raw results of every measurement quoted here are available at <https://github.com/jesusjuansalgado/skycell> under the MIT licence, together with the history of the measurements that led to the current defaults. All corpora but the real one are generated from seeds; the real one is drawn from the Gaia archive by a recorded query.

Acknowledgements

This work has made use of data from the European Space Agency (ESA) mission *Gaia* (<https://www.cosmos.esa.int/gaia>), processed by the *Gaia* Data Processing and Analysis Consortium (DPAC, <https://www.cosmos.esa.int/web/gaia/dpac/consortium>). Funding for the DPAC has been provided by national institutions, in particular the institutions participating in the *Gaia* Multilateral Agreement.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

References

- [1] Chilingarian, I., Bartunov, O., Richter, J., & Sigaev, T. 2004, in ASP Conf. Ser. 314, Astronomical Data Analysis Software and Systems XIII, ed. F. Ochsenbein, M. G. Allen, & D. Egret (San Francisco: ASP), 225
- [2] Delli Veneri, M., & Salgado, J. 2026, egeria: an IVOA TAP 1.1 server for the SKA Observatory, Zenodo, doi:10.5281/zenodo.22845690
- [3] Demleitner, M., Neves, M. C., Rothmaier, F., & Wambsganss, J. 2014, Astron. Comput., 7–8, 27
- [4] Dowler, P., Rixon, G., Tody, D., & Demleitner, M. 2019, IVOA Recommendation: Table Access Protocol Version 1.1
- [5] ESA 1997, The Hipparcos and Tycho Catalogues, ESA SP-1200

- [6] Fernique, P., Boch, T., Donaldson, T., et al. 2014, IVOA Recommendation: MOC – HEALPix Multi-Order Coverage map Version 1.0
- [7] Gaia Collaboration, Prusti, T., de Bruijne, J. H. J., et al. 2016, *A&A*, 595, A1
- [8] Górski, K. M., Hivon, E., Banday, A. J., et al. 2005, *ApJ*, 622, 759
- [9] Gray, J., Nieto-Santisteban, M. A., & Szalay, A. S. 2006, The Zones Algorithm for Finding Points-Near-a-Point or Cross-Matching Spatial Datasets, Microsoft Research Technical Report MSR-TR-2006-52
- [10] Koposov, S., & Bartunov, O. 2006, in ASP Conf. Ser. 351, *Astronomical Data Analysis Software and Systems XV*, ed. C. Gabriel, C. Arviset, D. Ponz, & S. Enrique (San Francisco: ASP), 735
- [11] Kunszt, P. Z., Szalay, A. S., & Thakar, A. R. 2001, in *Mining the Sky*, ed. A. J. Banday, S. Zaroubi, & M. Bartelmann (Berlin: Springer), 631
- [12] Moon, B., Jagadish, H. V., Faloutsos, C., & Saltz, J. H. 2001, *IEEE Trans. Knowl. Data Eng.*, 13, 124
- [13] Ortiz, I., Lusted, J., Dowler, P., et al. 2008, IVOA Recommendation: Astronomical Data Query Language Version 2.0
- [14] Rots, A. H. 2007, IVOA Recommendation: Space-Time Coordinate Metadata for the Virtual Observatory Version 1.33
- [15] Salgado, J., González-Núñez, J., Gutiérrez-Sánchez, R., et al. 2017, *Astron. Comput.*, 21, 22
- [16] Salgado, J., et al. 2020, in ASP Conf. Ser. 522, *Astronomical Data Analysis Software and Systems XXVII*, ed. P. Ballester, J. Ibsen, M. Solar, & K. Shortridge (San Francisco: ASP), 307